

Dependable Distributed Systems

Report

API-Driven Control of IOTA Hornet Nodes in a Dockerized Environment

Gaith Adra
Jasper Ebbelaar
Hyeonhui Lee

Course: "Softwareprojekt: Verteilte Systeme"
Course-ID: 19315312
Lecturer: Justus Purat

Institute of Computer Science, Freie Universität Berlin, Germany

July 16, 2025

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Outline of Contribution	3
2	Background	4
2.1	About the Project	4
2.2	Frameworks	4
3	Concepts	5
4	Workflow	8
4.1	Tasks	8
4.2	Problems & Solutions	9
4.3	Communication in the group	11
5	Results	12
5.1	Requirements and Used Versions	12
5.2	User Manual	13
5.2.1	Web API	13
5.2.2	Command Line API	14
5.2.3	How to use	14
6	Conclusion	17
6.1	Discussion of Results	17
6.2	Limitations	17
6.3	Future Work	18

CHAPTER 1

Introduction

1.1 Motivation

As decentralized networks and distributed ledger technologies (DLTs) become more popular, managing and running these systems efficiently presents new challenges. IOTA, with its unique Tangle architecture, has gained attention as a promising DLT. However, managing IOTA nodes, particularly Hornet nodes, can be complex, especially when dealing with multiple nodes that need to communicate with each other. This challenge grows when these nodes are containerized using Docker, as it can become time-consuming and error-prone to manage the containers, networks, and node connections manually.

This project is motivated by the need to simplify the management of multiple Hornet nodes in a Dockerized environment. Docker makes it easier to package and run applications in isolated containers, but managing large numbers of containers, especially in the context of a distributed system like IOTA, requires automation. In particular, ensuring that Hornet nodes can communicate with each other and maintain the proper network connections is not an easy task without a streamlined approach.

The goal of this project is to develop an API that simplifies the process of managing Hornet nodes and their networks. This includes features like creating and stopping nodes, setting up Docker networks, and connecting or disconnecting nodes from each other. By automating these tasks, the project aims to reduce human error and improve the ease of managing IOTA-based networks.

This solution is especially useful for those who want to run a large number of IOTA nodes, but don't want to spend too much time on manual configuration. By providing both a command-line interface and a web interface, users can choose the method that best fits their needs—whether they're developers looking to automate tasks or operators needing a simple way to manage the network. Ultimately, this project helps make the process of running a decentralized IOTA network more efficient and accessible to a wider audience.

1.2 Outline of Contribution

This project simplifies the management of IOTA Hornet nodes and their Docker environments by creating an API that automates tasks such as node creation, termination, and network management. The goal is to make managing IOTA nodes more efficient and user-friendly, reducing the complexity of working with distributed ledger technologies and Docker environments.

The API provides three main functions: node management (creating and stopping nodes), network management (creating and removing Docker networks), and node peering (connecting and disconnecting nodes within the same network). By automating these tasks, the project reduces manual work and the chance of errors, making it easier to manage multiple Hornet nodes at once.

Users can interact with the system through two interfaces. The command-line interface is designed for technical users or developers, while the web interface offers a more intuitive and accessible way for non-technical users to manage nodes and networks.

The main contribution of the project is its automation, which improves the efficiency and reliability of managing IOTA networks. By streamlining node and network management, the project allows users to easily deploy and maintain large-scale distributed systems. This solution makes it easier for developers and operators to scale IOTA-based networks while reducing complexity and manual intervention.

In summary, this project provides a simple yet powerful way to manage IOTA Hornet nodes and networks using an automated API. With both command-line and web interfaces, it makes the management of IOTA nodes more efficient and user-friendly.

CHAPTER 2

Background

2.1 About the Project

The goal of the project was to implement an API for interacting with IOTA Hornet nodes. The API is implemented in form of a WebAPI using Django and an API for the CLI. The functionality of the Hornet nodes was provided, with an example implementation in which the nodes were running in a private tangle. The API provides the functionality to start/end Hornet nodes, connect/disconnect Hornet nodes and to create/delete Docker networks.

2.2 Frameworks

The frameworks which were used in this project are Docker and Django.

Docker:

- Docker is a software which allows to run programs in containers
- The Hornet nodes and its modules are containers
- The private tangle each node runs in is a Docker network
- The overlay network which the nodes get added to are Docker networks

Django:

- Django is a Webframework
- We use it to provide a Webinterface for our API
- The Webinterface calls the functions from our API

CHAPTER 3

Concepts

After deciding what the goals of this project are, the next step was to imagine how the implementation would look like; which components there will be, how they're all connected and how they interact with one another without any conflicts. In this chapter, the architecture of the software project, as well as its components are provided with all the relative information.

Since our goal is to create an API that provides certain functionalities, more on that in Chapter 4, the plan was to run it on an accessible server (in our case and for the sake of developing the program, localhost was used) and execute our commands as easily as possible. To that end two communication methods with the API were implemented. The first being directly through the command line prompt, and the other, the more sophisticated and more user friendly to use, was through a website that was created for this project. The developed API, through its functions, communicates with Hornet nodes that are operated by Docker as containers. This means that the core functionalities of the implemented functions is to make calls to other remote APIs (Hornet, Docker) to achieve a specific goal, such as starting a new Hornet node or shutting a node down. Of course, these functions are written and customized in a way to make them fit perfectly for the use case of this project and achieve the specified goals, while maintaining a good level of user friendliness.

Now that high level components were introduced, it's time to discuss the implemented functions themselves in detail. But before explaining the functions, it is very important to note that there was a process to create docker files for a total of 100 Hornet nodes, each with their own uniquely assigned number (0-99). These are the Hornet nodes that are used throughout the entirety of this project. The functions that comprise the API are split into the following three categories:

- Category 1 handles the basic functions, on which the subsequent categories build upon. Here, the two main functions `create_node(node_number)` and `stop_node(node_number)`, as well as the helper function `container_state(container_name)` are explained.

The first main function, `create_node`, simply starts the Docker container of a specified Hornet node. The second main function, `stop_node`, simply stops the Docker container of a specified Hornet node that is already running. All Hornet nodes that are used in this project have been generated by a script (more on that in Chapter 4). The helper function, `container_state`, is called from within the mentioned main functions to check for the state of the Docker containers of the Hornet nodes that the user wants to create or stop.

- Category 2 handles the creation of virtual Docker networks, where running Hornet nodes can peer through. Here, the functions `create_network(network_name)` and `stop_network(network_name)` are explained. The first function, `create_network`, creates a Docker network that is named using the parameter. This network name is important for the functions of Category 3. The second function, `stop_network`, removes the network with the specified name if it has already been created.
- Category 3 handles the connection of running Hornet Docker containers to a network that has been created (from Category 2) and their disconnection, as well as the peering of Hornet nodes with each other and their disconnection. Here, there are a total of 6 functions, two of which are main functions and the other four are helper functions. The helper functions are `connect_containers(network_name, container_names)`, `disconnect_containers(network_name, container_names)`, `get_ip(network_name, container_name)`, and `get_id(container_name)`. The first two functions connect the Docker containers of the Hornet nodes to a network (from Category 2) and are called from within the main functions `connect_nodes(network_name, host_node, node)` and `disconnect_nodes(network_name, host_node, node)` respectively (only Hornet nodes within the same Docker network are able to peer with one another). The main functions are used to create peer connections between the Hornet nodes within a Docker network or disconnect them. The first main function, `connect_nodes`, starts by connecting the containers of two Hornet nodes to a Docker network. And now since these Hornet nodes are within a new network, they are assigned new IP addresses. IP addresses as well as node IDs are required to manage the peering of nodes. IP addresses are retrieved by querying the Docker network, which is what the function `get_ip` does, and node IDs are retrieved by querying the container of that node itself, which is what the function `get_id` does. Knowing the IP addresses and the node IDs enables sending a POST request to the Hornet API to establish connections/ disconnections between different nodes. In the case of connecting nodes, after the POST request has been successfully sent to the Hornet API, the "node" parameter Hornet node gets restarted, because some of the Docker containers, that are configured within the Docker configuration file of that node (like the coordinator), need to be shutdown in order for those Hornet nodes to be able to peer with each other. After it starts, with the `create_node` function from Category 1, this node will be peered to the "host_node" Hornet node parameter. And in the function `disconnect_nodes`, a POST request is sent to the Hornet API to unpeer the "node" parameter from the "host_node" parameter. After that, the "node" parameter's Docker container gets disconnected from

the Docker network it used to share with the "**host_node**" and is completely shut down and restarted.

CHAPTER 4

Workflow

4.1 Tasks

Now that the goals and the approach have been explained, it is time to discuss the development of this project in detail. In the chapter order of implementing the tasks, the problems that occurred, and their solutions are explained.

Starting with the order of the tasks that our team went through, first we split the project into five milestones. We worked exactly two weeks for each milestone, and the work included researching the theory of how some components function (IOTA, Hornet) and then we researched how the different components actually work as platforms and programs (IOTA, Hornet, Docker, Django). And only after fully understanding what we are dealing with and how, we moved on to implement the functionality of a given milestone. For the first milestone, we had the most extensive research sessions on the topics of IoT in distributed systems, IOTA and Hornet. However, on top of the research our goal for that first milestone was to establish a Django server, that would act as an API, with a website, that the user eventually would use. We also customized the layout and the looks of the webpage so it doesn't look boring and stale for the users' eyes. For the second milestone, our goal was to develop the first two functions of the API, create node and stop node. because as mention in Chapter 3, it was mandatory that we start with these two functions for our API, because almost all subsequent functions rely on being able to manage the Hornet nodes. To that end, first we spent a good amount of time researching how Docker works and how containers can be managed. Then, we wrote our own script to generate the configuration files for Docker containers of 100 Hornet nodes; each with their unique Alias, IP addresses and ports. Each configuration file contains configurations for essential Docker containers, and each of these should have their own unique IP addresses and ports (all ports assigned are internal and are connected to external ones). After we've acquired the nodes that we can work with for the rest of the project, we wrote the code for the functions `create_node` and `stop_node` along with their helper function `container_state`, as mentioned in Chapter 3. And as a final step

for each milestone, extensive tests and debugging were made. The third milestone was to create and stop Docker networks in addition to implementing the functionalities of connecting Hornet Docker containers to a created network and disconnecting them. At the beginning of this milestone we took our time to decide on how exactly we want to allow connections to be built between the Hornet Docker containers. After that we implemented the functions `create_network`, `stop_network`, `connect_containers`, and `disconnect_containers`, which were fairly easy to implement and test after the brainstorming phase of the milestone. The fourth, and most difficult, milestone was to implement functionalities to peer and unpeer Hornet nodes. Here again, we had to extensively research how this can be done from a technical perspective and in the end we found out that we can use the Hornet API to establish peer connections between the Hornet nodes, by sending a POST request provided with IP addresses, ports and Aliases of the nodes we want to peer (an example of this POST request can be seen in Chapter 5). Using the Hornet API was again our approach for the function `disconnect_nodes` (an example of this POST request can be seen in Chapter 5). A LOT of testing and debugging went into this phase of the project. For the fifth and final milestone we went through our code again and handled all errors and cleaned up the code. Beside that we connected the API functionalities to our website (this was all done using Django). Before completing this milestone, the command line was the only way to use and test the functions, but after updating our website and making a new python module with the same core functionalities as the previously implemented functions, that was compatible with the website approach, we were able to run our functions through it. Now users can either use the website, for a clean and comfortable experience, or use the command line for specific debugging or continue developing the project.

4.2 Problems & Solutions

During our work on the project we encountered various problems:

1. We were not used to work with Github, so there were problems with setting it up, untracked files, deleting of files which were not edited and rolling back to an older commit
2. Familiarizing with Django; We never used Django before so there were problems regarding the workflow within Django, understanding how views work, how to debug with Django and a problem instability regarding the error/success messages on the webinterface
3. If a node crashes other containers don't shut down and the directory privateDB is not deletable
4. When nodes get peered one of them crashes
5. If the Docker overlay network is removed nodes cannot be restarted
6. Nodes get a new IP-address when added to the overlay network

7. Peers need to be added manually on every node
8. The nodes do not instantly crash when peered, it sometimes takes around 20 seconds until they crash
9. Our original api.py file did not work with the Django server
10. Understanding what IOTA and Hornets are in general
11. Running the bootstrap.sh script requires sudo rights on Linux
12. We used different Operating systems(Linux, Windows and MacOS), which leads to different problems
13. Figuring out how to peer nodes

Solutions:

1. Github needs an access token instead of a password when used in the CLI, most other problems could be solved by cloning the repository again
2. The tutorial on the Django webpage is a good start to learn Django, after the tutorial we had a working server which we could modify for our project
3. If a node crashes the cleanup.sh script needs to be used to stop all containers of a node
4. We couldn't prevent the node to crash, instead the node will be restarted
5. Need to keep track of which nodes and networks are running
6. There is a command to find out what the IP address of a container in a specific Docker network is
7. The Hornet nodes provide an API themselves which give a command to add peers to a node
8. After the node gets added as a peer the script waits for 30 seconds until it restarts a node
9. We created a webapi.py which has changes regarding the error/success messages and changing to the node directory
10. After some research, explanations by Justus and a lot of testing we figured out how it works
11. On Linux the sudo password has to be entered into the CLI
12. The script checks if it is executed on Linux or MacOS, on MacOS sudo is not necessary; We decided to not bother about getting it to run on Windows, there were too many problems
13. This was the biggest challenge of the project and took a lot of time testing various approaches to peering two nodes, in the end we found a way to peer two hornets

4.3 Communication in the group

The communication in the group went very well. We used Discord and Whatsapp for communicating. On Whatsapp we used a group for text messaging. Except during the Christmas time, we had a weekly meeting in a Discord call to talk about the tasks, our progress and preparing the presentations for our sprints.

CHAPTER 5

Results

Now that the implemented functionalities have been discussed in detail in the previous chapters, it is time for the result discussion. The API works as previously explained and the source code for the entire project can be found at <https://github.com/Gheus11/SWP-Distributed-Systems>. In this chapter information about the requirements and usage of our code is specified; starting with Python library requirements, then information on how to setup and run the API, and finally some details on how exactly to use the implemented functionalities.

5.1 Requirements and Used Versions

- **API:**
 - Django 5.1.3
- **Functionalities:**
 - docker 7.1.0
 - requests 2.32.3
- **Node Config File Generation:**
 - bip_utils 2.9.3 (works with Python versions 3.7 to 3.12)
 - cryptography 3.4.8
 - base58 2.1.1

The rest of the libraries used are all built in libraries that don't need to be downloaded on their own.

5.2 User Manual

You can operate a total of 100 Hornet nodes. Each Hornet node is named hornet-00 through hornet-99.

5.2.1 Web API

	Description	Input
Create Node	Create a hornet node	The number of the Hornet node you wish to create. It must be an integer between 0 and 99.
Stop Node	Terminate a hornet node	The number of the Hornet node you wish to terminate. It must be an integer between 0 and 99.
Create Network	Create a Docker overlay network	The name of the network you want to create
Stop Network	Terminate a Docker overlay network	The name of the network you want to terminate
Connect Nodes	Set a host node for the given Docker overlay network and connect the second node to the host node.	Input 1: the name of the hornet you want to set as the host hornet, Input 2: the name of the hornet you want to connect to the host hornet, Input 3: the name of the network that contains the host hornet.
Disconnect Nodes	Disconnect the second node from the host node in the given network.	Input 1: the name of the host hornet, Input 2: the name of the hornet you want to connect to the host hornet, Input 3: the name of the network that contains the host hornet.

5.2.2 Command Line API

	Description	Usage
create_node	Create a hornet node	python api.py create_node {HORNET_NUMBER}
stop_node	Terminate a hornet node	python api.py stop_node {HORNET_NUMBER}
create_network	Create a Docker overlay network with a custom name	python api.py create_network {NETWORK_NAME}
stop_network	Terminate a Docker overlay network	python api.py stop_network {NETWORK_NAME}
connect_nodes	Set a host node for the given Docker overlay network and connect the following nodes to the host node.	python api.py connect_nodes {NETWORK_NAME} {HOST_HORNET_NAME} {HORNET_NAME_1} {HORNET_NAME_2} ...
disconnect_nodes	Disconnect the following nodes from the host node in the given network.	python api.py disconnect_nodes {NETWORK_NAME} {HOST_HORNET_NAME} {HORNET_NAME_1} {HORNET_NAME_2} ...

5.2.3 How to use

Config generator:

In the directory `"../DjangoServer/nodes"` are 100 directories with configured hornet nodes in it. If changes to the config want to be made there is a config generator for 100 nodes. In `"../config_generation/docker-compose_template.yml"` changes can be made to the template. The generator can be used on the CLI by moving to the directory `"../config_generation"` and executing the command `"python docker-compose_generator.py"`. Then 100 new node configs will be generated.

Web API:

To use the Web API the Django server has to be started first. To start the server first move to the directory `"../DjangoServer"` and use the command `"python manage.py run-server"`. The web interface can now be accessed by opening the URL `"http://127.0.0.1:8000/"`. There are 3 different users available to login: (username: user0 ; password: 0), (username: user1 ; password: 1) and (username: user2 ; password: 2). After logging in the dashboard of the web interface opens. On the dashboard the functions of the API can be used by filling the text field and pressing the button. If Linux is being used the sudo password has to be entered in the CLI when calling a function. After the sudo password

got accepted, it has not to be reentered for 5 minutes.

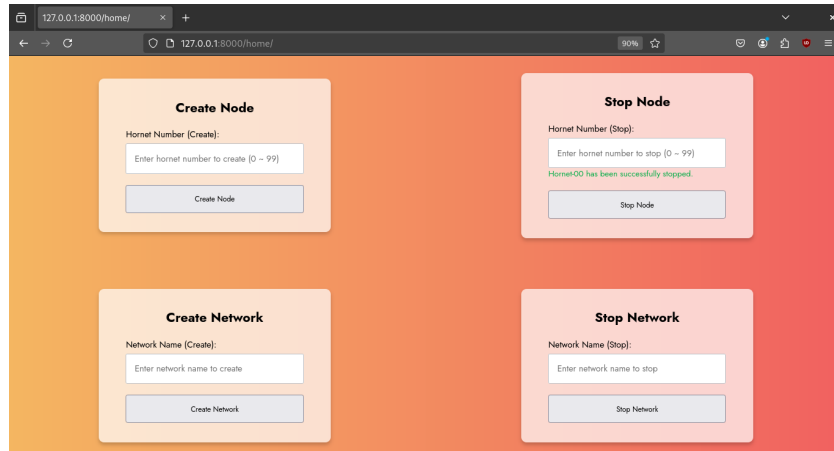


Figure 5.1: Web interface

Command Line API:

The command line API can be used by first moving to the directory `../DjangoServer/myapp` and there then the API functions can be used with the command `"python api.py function_name"`, where `function_name` is one of the functions from the table 5.1.2. On Linux the sudo password has to be entered.

Important:

For create and stop node only a number between 0 and 99 has to be entered. When using the connect and disconnect functions the Hornet names have to be the same as the name of the Docker container of the Hornet. An example to use connect nodes in the Command Line API is: `"python api.py connect_nodes testnet hornet-00 hornet-01"`.

How to access the Dashboard of a Hornet:

After a Hornet node got created the Dashboard of the Hornet can be accessed through a Browser. To open the Dashboard of a Hornet the address `"172.18.1XX.28:8081"` has to be opened, where XX has to be replaced with the number of the Hornet. The login credentials for every node are `"Username: admin Password: admin"`.

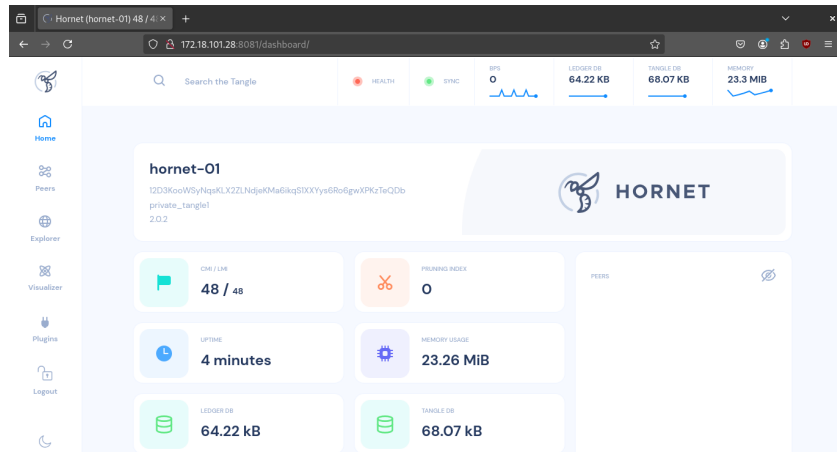


Figure 5.2: Dashboard of a Hornet node

CHAPTER 6

Conclusion

6.1 Discussion of Results

The results of this project show that the developed API successfully automates the management of IOTA Hornet nodes and Docker environments. The API allows users to create and stop nodes, manage Docker networks, and establish peer connections between nodes, significantly simplifying the process of handling multiple nodes within an IOTA network.

The implementation of both a command-line interface and a web interface enables users with varying technical expertise to interact with the system effectively. The automation reduces manual effort and the risk of errors, making node management faster and more efficient.

Based on these results, the problem stated in the introduction—automating the management of IOTA Hornet nodes to enhance efficiency and scalability—has been solved. The project provides a reliable and user-friendly solution to manage large-scale IOTA networks, making node creation, termination, and peering seamless. This was made possible by automating key tasks, integrating Docker for container management, and offering both command-line and web interfaces for user interaction. Therefore, the project successfully meets its objectives, solving the problem by reducing manual work and improving network management.

6.2 Limitations

One of the major limitations of our API lies in the process of connecting Hornet nodes. Each time a Hornet node is connected to a host node, the connected Hornet nodes are forced offline. This issue arises due to the absence of synchronization milestone between the nodes, causing problems with syncing and resulting in the guest node going offline. To mitigate this, we regenerate the guest node; however, the downside is that when the

node is recreated, the associated database containing critical Tangle information—such as coordinator, transactions, and milestones—is reset. Consequently, this leads to the loss of a fully functional Tangle, affecting the integrity of the network.

Another limitation exists in error handling. During the use of the web interface, users may encounter error scenarios that we did not anticipate or address. These unforeseen errors could lead to confusion for the user, as they may not be immediately clear or addressed properly within the system.

Additionally, there is a lack of notifications when users send requests through the web interface. This absence of feedback can create confusion and make it difficult for users to understand whether their request was successfully processed. Furthermore, if a user sends another request while one is still executing, overlapping processes can occur, leading to potential conflicts or unexpected behaviors.

Finally, the performance of the system is heavily dependent on the specifications of the user's computational resource. Running multiple Hornet nodes simultaneously can put a significant load on the system, requiring high-performance hardware. This limits the scalability of the system on machines with lower specifications, as handling a larger number of nodes demands substantial computational resources.

6.3 Future Work

When connecting two Hornet nodes, one major issue arises when the guest Hornet goes offline, making it impossible to maintain the integrity of the Tangle. This happens because the coordinator fails to find a common milestone between the two nodes. One possible solution is to adjust the configuration settings to prevent the Hornet node from forcibly going offline in such cases. While this approach may help mitigate the issue, it is not a complete solution for maintaining a fully functional Tangle. A more robust approach would involve integrating the coordinators and the Tangle logic of both nodes to ensure seamless synchronization. However, this presents a challenging task that requires careful planning and implementation.

Another important aspect to consider is user experience when background processes are running. It is crucial to inform the user when a task is being executed in the background to prevent confusion. This can be achieved by implementing a loading indicator or similar UI element to signal ongoing activity. Additionally, to avoid conflicts and ensure smooth operation, it is necessary to disable buttons or restrict user interactions while a process is still running. This prevents multiple simultaneous requests from being sent to the server, which could cause unintended issues.

Furthermore, improving error handling is essential for creating a more robust web interface. Various error scenarios should be anticipated and handled appropriately to ensure the system remains stable under different conditions. This includes providing clear error messages, implementing fallback mechanisms, and ensuring that failures do not lead to unexpected behavior. By addressing these aspects, the system can become more reliable,

user-friendly, and resilient against potential failures.

Declaration of independence

We hereby declare to have written this thesis on my own. We have used no other literature and resources than the ones referenced. All text passages that are literal or logical copies from other publications have been marked accordingly. All figures and pictures have been created by me/us or their sources are referenced accordingly. This thesis has not been submitted in the same or a similar version to any other examination board.